CS 341 Week 3 Lab Answer — Jeopardy

Category 1 — C & Pointers Review

$100
`int *p` points at the start of an array. On a machine where `int` is 4 bytes, how many bytes does `p + 1` move?

A. 1
B. 4
C. 8
D. Depends on the value stored at `p`

Answer: B — 4 bytes

$200
On a 64-bit machine, what are `sizeof(a)` and `sizeof(b)`?

```C/C++
char a[] = "cat";
char *b = "cat";
```

A. 3 and 3
B. 4 and 8
C. 3 and 8
D. 4 and 4

Answer: B — 4 and 8

$300
Writing to `a[0]` works; writing to `p[0]` crashes. In one word: what kind of memory does the string literal `*p` point to?

```C/C++
char a[] = "hi";
char *p = "hi";
```

Answer: read-only or text segment

$400

The caller's pointer never changes, because `s` is only a copy of it. What must the function parameter's type be so the caller's pointer does change?

```C/C++
void f(char *s) { s = "bye"; }
```

Answer: `char **`, pointer to a pointer, double pointer

$500

This should print all five numbers. It prints `10 20`. Find the bug and fix it.

```C/C++
void print_all(int arr[]) {
    for (int i = 0; i < sizeof(arr) / sizeof(arr[0]); i++)
        printf("%d ", arr[i]);
    printf("\n");
}

int main(void) {
    int nums[5] = {10, 20, 30, 40, 50};
    print_all(nums);
    return 0;
}
```

Answer: `sizeof(arr)` works only where `arr` is still an array, but arrays decay to pointers when passed to a function.

Note: `sizeof(arr) / sizeof(arr[0])` works only where the array is still an array. Passing an array to a function decays it to a pointer, so inside `print_all`, `arr` is an `int *`, not an `int[5]`. The array's length is simply not there anymore.

Fix: the length has to be passed explicitly. It cannot be recovered.

```C/C++
void print_all(int *arr, size_t n) {  // int arr[] means int * here anyway
    for (size_t i = 0; i < n; i++)
        printf("%d ", arr[i]);
    printf("\n");
}
...
print_all(nums, sizeof(nums) / sizeof(nums[0]));
```

Category 2 — Heap Memory & Memory Errors

$100
Which of these returns memory that is not guaranteed to be zeroed?

A. `calloc`
B. `malloc`
C. Both are zeroed
D. Neither is zeroed

Answer: B — malloc

$200
What is wrong with this code?

```C/C++
int *p = malloc(sizeof(int) * 4);
p[0] = 42;
free(p);
printf("%d\n", p[0]);
```

A. Nothing. Reading after `free` is safe, only writing is a problem
B. It is a use-after-free, which is undefined behavior
C. It is a double free
D. `p` needed to be set to `NULL` before the `printf`

Answer: B — use-after-free

$300
A function returns a pointer to an array declared inside itself. The array's memory is gone the moment the function returns.

What is that returned pointer called?

Answer: a dangling pointer

$400

What does this print?

```c
C/C++
char *label(const char *s) {
    static char buf[16];
    snprintf(buf, 16, "[%s]", s);
    return buf;
}
char *a = label("one");
char *b = label("two");
printf("a=%s b=%s\n", a, b);
```

Answer: a=[two] b=[two]

$500

This program prints **`widget 1 / widget 2`** correctly, and then it dies during cleanup. Every pointer in this program is freed exactly once in the source. So why does it still crash, and what is the proper fix?

```c
C/C++
typedef struct { char *name; int id; } Item;

Item *make_item(const char *n, int id) {
    Item *it = malloc(sizeof(Item));
    it->name = malloc(strlen(n) + 1);
    strcpy(it->name, n);
    it->id = id;
    return it;
}

void destroy_item(Item *it) {
    free(it->name);
    free(it);
}

int main(void) {
    Item *a = make_item("widget", 1);
```

```
    Item *b = malloc(sizeof(Item));
    *b = *a;
    b->id = 2;

    printf("%s %d / %s %d\n", a->name, a->id, b->name, b->id);

    destroy_item(a);
    destroy_item(b);
    return 0;
}
```

Answer: It's a double free, even though no pointer is freed twice in the text of the program.

Note: `*b = *a` is a shallow copy. It copies the struct's contents, which means it copies the name pointer, not the string it points at. After that line, `a->name` and `b->name` hold the same address: one heap block, two owners. `destroy_item(a)` frees that block. `destroy_item(b)` frees it again.

Fix: deep copy. `b` needs its own buffer.

```C/C++
Item *b = malloc(sizeof(Item));
b->name = malloc(strlen(a->name) + 1);
strcpy(b->name, a->name);
b->id = 2;
```

Category 3 — Processes & Memory Space

$100

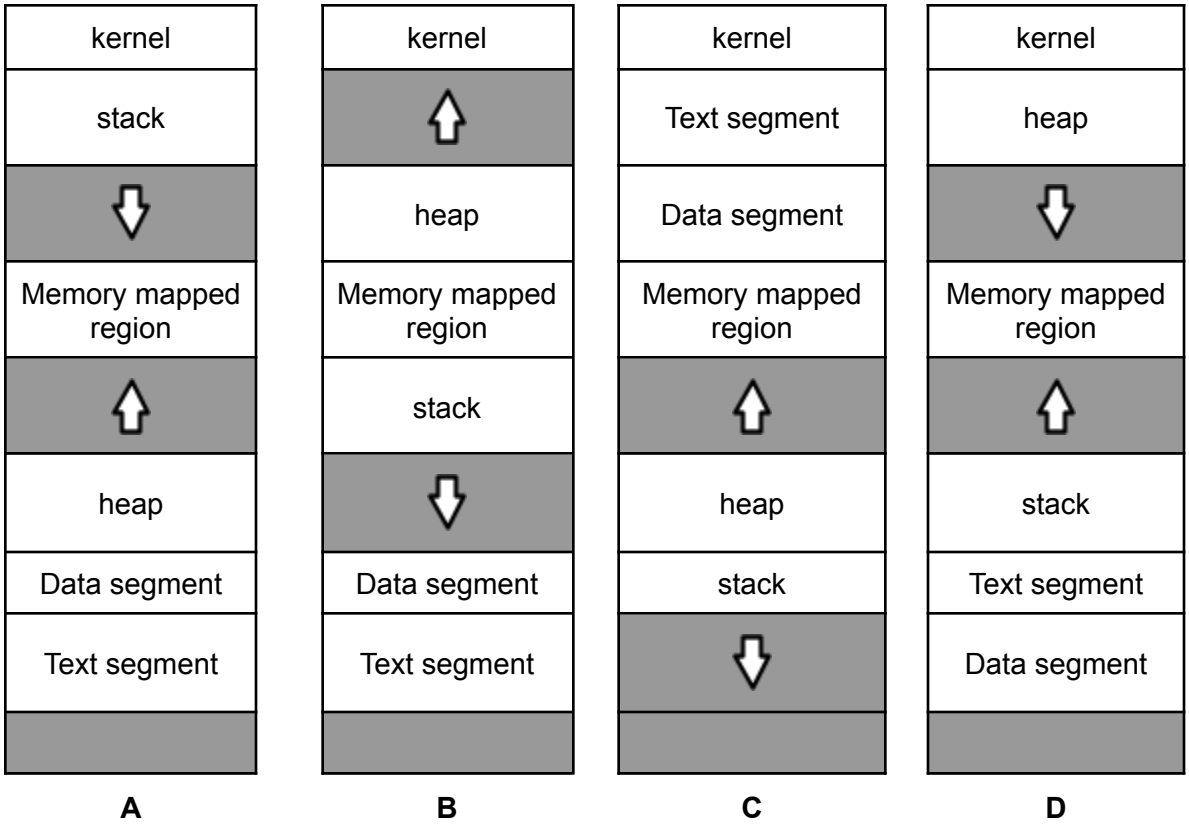
A process calls `fork()` and it succeeds. What does `fork()` return **in the child process**?

A. The child's own PID
B. 0
C. The parent's PID
D. -1

Answer: B — 0

$200

Which diagram accurately represents a process memory layout? (high address at top)



Answer: A

$300

After `fork()`, which of these does the child **not** get a copy of?

A. The parent's environment variables
B. The parent's signal handlers
C. The parent's pending signals
D. The parent's open file descriptors

Answer: C — pending signals

$400

What does this print?

C/C++

```c
int main(int c, char **v) {
    while (--c > 1 && !fork());
    sleep(c = atoi(v[c]));
    printf("%d\n", c);
    wait(NULL);
    return 0;
}
```

Shell

```
$ ./main 2 3 4 1
```

Answer: 1 2 3 4

$500

This is supposed to start three tasks and print three lines. How many lines does it actually print, what is the bug, and how do you fix it?

C/C++

```c
int main(void) {
    for (int i = 0; i < 3; i++) {
        pid_t pid = fork();
        printf("task %d started\n", i);
    }
    return 0;
}
```

Answer: 14 lines, from 8 total processes

Note: Parent and child both print, and both go around the loop again and fork again.

Pass	Processes Forking	Processes After	Lines Printed
`i=0`	1	2	2
`i=1`	2	4	4
`i=2`	4	8	8
		8 total	**14 total**

Fix: use the `fork()` return value to split the two paths, and make the child leave.

```C/C++
for (int i = 0; i < 3; i++) {
    pid_t pid = fork();
    if (pid < 0) { perror("fork"); exit(1); }
    if (pid == 0) {              // child only
        printf("task %d started\n", i);
        exit(0);                 // child must not loop again
    }
}
for (int i = 0; i < 3; i++) wait(NULL);   // parent reaps
```

Category 4 — GDB and Valgrind

$100
In GDB, you are stopped on a line that calls a function. Which command runs that whole line and stops on the next line, without going inside the function?

A. `step`
B. `next`
C. `continue`
D. `finish`

Answer: B — next

$200
Valgrind reports a block as "**definitely lost.**" What does that mean?

A. The program read past the end of the block
B. The block was freed twice
C. The block was allocated, never freed, and no pointer to it survives
D. The block was read before it was initialized

Answer: C — a genuine leak with no surviving pointer.

$300
You run `valgrind ./myprog` and see a leak summary reporting 39 bytes definitely lost, but no location. What option do you add to find out **where**?

```
Shell
==511== HEAP SUMMARY:
==511==     in use at exit: 39 bytes in 2 blocks
==511==   total heap usage: 4 allocs, 2 frees, 4,140 bytes
allocated
==511==
==511== LEAK SUMMARY:
==511==    definitely lost: 39 bytes in 2 blocks
==511==    indirectly lost: 0 bytes in 0 blocks
==511==      possibly lost: 0 bytes in 0 blocks
==511==    still reachable: 0 bytes in 0 blocks
==511==         suppressed: 0 bytes in 0 blocks
```

Answer: `--leak-check=yes` (or `--leak-check=full`)

$400

This program segfaults. The call on line 23 completes fine, but it crashes somewhere in the call on line 24. You want to step through **only the failing call**. How would you set a breakpoint in GDB?

```C/C++
 5  typedef struct { char *name; int qty; } Item;
 6
 7  Item *find_item(Item *items, int n, const char *name) {
 8      for (int i = 0; i < n; i++) {
 9          if (strcmp(items[i].name, name) == 0)
10              return &items[i];
11      }
12      return NULL;
13  }
14
15  void restock(Item *items, int n, const char *name, int amount)
{
16      Item *it = find_item(items, n, name);
17      it->qty += amount;
18  }
19
20  int main(void) {
21      Item stock[3] = { {"bolt", 10}, {"nut", 4}, {"washer", 25}
};
22
23      restock(stock, 3, "nut",   6);
24      restock(stock, 3, "cable", 5);
```

Answer: `break restock if amount == 5` **(condition 1** `amount == 5` **after a plain break)**

$500

append is correct. **`length`** is correct. It still segfaults. Read the GDB backtrace. What does it tell you, and what is the bug?

C/C++

```c
typedef struct Node { int v; struct Node *next; } Node;

Node *append(Node *head, Node *n) {
    if (head == NULL) return n;
    Node *cur = head;
    while (cur->next != NULL) cur = cur->next;
    cur->next = n;
    return head;
}

int length(Node *node) {
    if (node == NULL) return 0;
    return 1 + length(node->next);
}

int main(void) {
    Node *a = make_node(1), *b = make_node(2), *c = make_node(3);
    Node *list = NULL;
    list = append(list, a);
    list = append(list, b);
    list = append(list, c);
    list = append(list, a);
    printf("length = %d\n", length(list));
}
```

```shell
Program received signal SIGSEGV, Segmentation fault.
(gdb) bt 8
#0  length (n=<error reading variable: Cannot access memory at
0x7fffff7feff8>)
#1  length (node=0x5555555592c0) at list.c:23
#2  length (node=0x5555555592a0) at list.c:23
#3  length (node=0x5555555592e0) at list.c:23
#4  length (node=0x5555555592c0) at list.c:23
#5  length (node=0x5555555592a0) at list.c:23
#6  length (node=0x5555555592e0) at list.c:23
#7  length (node=0x5555555592c0) at list.c:23
...
#262092 main () at list.c:37
```

Answer: The list is circular, so `length` recurses forever.

Note: Read the frame arguments. Three addresses repeat forever: **`92c0`**, **`92a0`**, **`92e0`**, **`92c0`**, **`92a0`**, **`92e0`**… The recursion is walking the same three nodes over and over. The last **`list = append(list, a);`** append walks to the tail, **`c`**, and sets **`c->next = a`**. Now **`a`** → **`b`** → **`c`** → **`a`**.

Fix: don't re-append an existing node.

Category 5 — Fork-Exec-Wait

$100
What must the last element of **`args`** be?

```C/C++
char *args[] = {"ls", "-l", ??? };
execvp("ls", args);
```

A. `""`
B. **`NULL`**
C. **`"ls"`**
D. The argument count

Answer: B — `NULL`

$200
This prints **`hi` twice**. Which change fixes it?

```C/C++
printf("hi");
pid_t pid = fork();
```

A. **`fflush(stdout);`** between the **`printf`** and the **`fork`**
B. **`fflush(stdout);`** in the child only, right after the **`fork`**
C. **`fflush(stdout);`** in both processes right before they return
D. Move the **`fork()`** above the **`printf()`**

Answer: A — flush buffer before you fork

$300
`"ls"` appears twice. The first one is the file path to load. What does the second one become inside the new program?

```C/C++
execlp("ls", "ls", "-l", NULL);
```

Answer: `argv[0]`

$400
A parent forks a child. Two scenarios:
(a) The child exits. The parent never calls **`wait`** and keeps running. What is the child now?
(b) Instead, the parent exits first while the child is still running. The child becomes an orphan and gets re-adopted. What is the PID of the new parent process?

Answer: (a) a zombie process (b) PID 1

$500
This is meant to launch 100 Ruby scripts in parallel and wait for them. On a machine where it **cannot** find **`ruby`**, it becomes a fork bomb. Why, and how would you fix it?

```C/C++
#define PROC 100
int main() {
    pid_t processes[PROC];
    for (int i = 0; i < PROC; ++i) {
        processes[i] = fork();
        if (!processes[i]) {
            execlp("ruby", "ruby", "file.rb", (char*) NULL);
        }
    }
    for (int i = 0; i < PROC; ++i) {
        waitpid(processes[i], NULL, 0);
    }
}
```

Answer: `execlp` only returns if it fails. The failed child falls through and keeps running the loop, forking its own children.

Note: On success the child is replaced by ruby and never runs the loop again. But if the exec fails, **`execlp`** returns, so the failed child falls through and keeps running the loop. It forks 100 children of its own. Each of those also fails to exec and falls through until the machine dies.

Fix: the child must exit after a failed exec.

```C/C++
if (!processes[i]) {
    execlp("ruby", "ruby", "file.rb", (char*) NULL);
    exit(1);           // stops the bomb: child must exit
```

```
}
```